

dSIBP 用户手册

Su Yingze^a

^a*Institute not specified*

E-mail: yingze.su@outlook.com

ABSTRACT: 本手册说明dS IBP package 的费曼规则、massive/massless 指标约定、time 与loop-momentum IBP、独立变量微分方程seed、EOM/缩并canonical、拓扑输入、公开API、Kira 接口和tree vertex family。公开topology 只使用严格的vertices/lines Association, 顶点以vertexType 和externalLegEnergy 定义外腿指数; full 模式使用J[aList,linePacks,ispList], time-only 模式使用J[sectorKey,timeShifts,stateBits]。用户显式分别声明任意命名的loopExternalMomenta 与independentExternalMomenta, 程序验证结构圈数、routing、声明完备性和动力学坐标, 不自动猜角色。package 只生成、序列化 and 验证关系, 不运行reduction。

Contents

1	快速开始与当前边界	2
2	完整例子: mix bubble+tree	4
3	SK 费曼规则到积分族	7
4	按mode 固定的三参数积分表示	8
5	Massive 导数、EOM 与缩并	9
6	Massless 双端点quotient 与time-IBP	11
7	Topology、标量积、ISP 与外腿能量	14
8	IBP 生成元与公开接口状态	20
9	Seed、linearData 与serializer	21
10	标准package 与初始化	25
11	运行消息与进度	26
12	Kira 结果取回到微分方程	27
13	Tree vertex family	30
14	常见错误与当前限制	31
A	公开函数汇总	32

1 快速开始与当前边界

本package 生成dS 圈图的canonical IBP seed, 并把关系转换为后端中立的linearData。它支持topology-driven 的任意圈数、任意拓扑以及massive/massless 混合内线; 唯一加载入口是发布目录内的标准模块树:

```
dSIBP/  
  dSIBP.m  
  Kernel/
```

本package 使用严格的干净输入schema, 并保留标准context、初始化metadata、完整time/loop-momentum 生成元、massive EOM、共同-theta odd-subset contact、contact 可达sector、跨sector coincident canonical、Kira serializer/importer 以及DE/scaling 检查。full 模式保留双端点三槽line pack; time-only 使用显式sector key、compact 时间幂和building-block 状态位列表。

在新kernel 中把发布目录加入\$Path 后加载:

```
packageDir = FileNameJoin[{Directory[], "dSIBP"}];  
If[!MemberQ[$Path, packageDir], AppendTo[$Path, packageDir]];  
Needs["dSIBP`"];
```

该入口提供dSIBP` 公开context。Examples/ 内的每个示例都通过Examples/load_current_package.wl 使用同一个入口, 因此不需要调用方设置路径。

全部模块成功加载后, 每个Wolfram kernel 只显示一次简洁引用提醒; Notebook 中的arXiv 号是可点击链接, headless kernel 同时保留完整URL。使用dSIBP 开展研究时, 建议引用:

1. Jiaqi Chen and Bo Feng, "Towards Systematic Evaluation of de Sitter Correlators via Generalized Integration-By-Parts Relations", [arXiv:2401.00129](#).
2. Jiaqi Chen, Bo Feng and Yi-Xiao Tao, "Multivariate hypergeometric solutions of cosmological (dS) correlators by d log-form differential equations", [arXiv:2411.03088](#).
3. Jiaqi Chen, Bo Feng, Zhehan Qin and Yi-Xiao Tao, "Loop integrals in de Sitter spacetime: The parity-split IBP system and d log-form differential equations", [arXiv:2604.14549](#).
4. dSIBP package paper, arXiv identifier pending.

程序包论文尚未提供arXiv 号, 因此本手册不为它生成链接或书目信息。

推荐工作流为

1. 输入顶点、内线、圈/外动量基、ISP、零点和必要的数值规则;
2. 用DSInit 验证输入、ISP 坐标、函数系统与contact-reachable sectors;
3. 用DSSeeds 生成普通canonical seed batch, 并从稳定字段"allSeeds" 或DSAllSeeds 取得全部sector/generator 的一维general 模板; massive 完整遍历端点四态, masslessFull 只生成 $n_2 = 0$ 的00/10 quotient representatives;

4. 模板层立即执行EOM/symmetry/canonical; 再用唯一入口DSGenerateIBP 显式展开连续指标;
5. 用DSLlinear 把撒点后的canonical batch 构造成后端中立linearData; 必要的小规模数值规则只在这一层代入;
6. 按需调用DSKiraExport; 用户在package 外运行reduction;
7. 有完整外部结果时用DSKiraImport -> DSDE -> DSScaleCheck 取回并验证。

解析seed 只允许小规模生成。禁止默认展开整族解析IBP, 也禁止为了验证做宽范围撒点。所有WolframScript 检查必须显示消息运行, 不能用Quiet 掩盖问题。

给定下文的case 后, 最短的可执行调用链是

```
context = DSInit[case];
templateData = DSSeeds[context,GenerateShrinkSectors->True];
allSeeds = DSAllSeeds[templateData];
seeds = DSGenerateIBP[allSeeds,{0,0}];
linearData = DSLlinear[seeds,context];
{context["status"],templateData["dSIBPStatus"],
 seeds["dSIBPStatus"],
 linearData["dSIBPStatus"]}
```

每个批量接口都返回Association; 必须先检查"status", 再读取方程或传给下一阶段。

发布目录Examples/ 提供六个应用例子:

```
01_mixed_bubble_workflow.wl
02_function_system_hankel.wl
03_single_massive_sunrise/main.wl
04_pure_massive_bubble_closed_loop/main.wl
05_tree_two_vertex_time_ibp/main.wl
06_mix_bubble_tree/main.wl
```

建议先运行06_mix_bubble_tree/main.wl; 它展示显式动量角色、ssij/sEe 初始化、参数重定义、总导数和cycle/fixed line 数据。两条cycle propagator 分别取 $b[e]+n[e,1]+n[e,2]==0 \pmod{2}$ 的全偶子空间; fixed bridge 不属于圈积分, 不进入parity constraints。03_single_massive_sunrise/main.wl 使用共同外腿能量kE、圈外动量kL、两个在massless-line 交换下互换的ISP, 以及顶点/平行线symmetryRules; 它只生成全部contact-reachable sector 的general IBP seeds 和{ss11,kE} general 参数微分算符。连续指标保持符号, 不进入连续撒点、linearData、Kira、DE 或scaling。

01_mixed_bubble_workflow.wl 只在内存中构造pre-reduction plan; 03 与05 不运行Kira, 05 也不写初始化目录。04_pure_massive_bubble_closed_loop/main.wl 展示外部Kira 回读、DE 与scaling; 06_mix_bubble_tree/ 同时给出其81-master 完整流程的轻量输入/结果摘要。examples 不含独立expected, 也不启动reduction。真实输入工作树由Windows Wolfram 写入仓库外目录, Kira 2.3 在WSL 的对应目录运行; 发布副本只保留可读输入与摘要, 已删除init/、kira/、database、save、日志、reduction table、cache 和DE 运行目录。

2 完整例子: mix bubble+tree

2.1 先按独立相位能量走通基准流程

考虑三个时间顶点。v1,v2 之间的bubble 有两条cycle lines, 动量分别为l1与l1+k1+k2; v2,v3 之间有一条无圈bridge line, 动量为k1+k2。v1 的另一侧外腿也携带k1+k2, v3 有两条外腿k1,k2。输入中的加减法始终是矢量线性组合:

$$|k_1 + k_2| = \sqrt{\text{sp}(k_1 + k_2, k_1 + k_2)} \neq |k_1| + |k_2| \quad (2.1)$$

(一般运动学下)。line 1 是唯一massive h cycle line, nu1 是它的Hankel 指标; line 2 是massless exponential cycle line, line 3 是massless exponential bridge。E1,E2,E3 是三个顶点指数相位的独立标量。package 不从传播子动量或nu1 推断相位变量。

完整核心输入为

```
case = <|
  "name" -> "mixBubbleTreeK1K2",
  "vertices" -> {
    <|"id"->v1,"vertexType"->"+", "externalLegEnergy"->E1|>,
    <|"id"->v2,"vertexType"->"+", "externalLegEnergy"->E2|>,
    <|"id"->v3,"vertexType"->"+", "externalLegEnergy"->E3|>
  },
  "lines" -> {
    <|"id"->1,"endpoints"->{v1,v2}, "momentum"->l1,
      "nu"->nu1, "massType"->"massive"|>,
    <|"id"->2,"endpoints"->{v1,v2}, "momentum"->l1+k1+k2,
      "massType"->"massless"|>,
    <|"id"->3,"endpoints"->{v2,v3}, "momentum"->k1+k2,
      "massType"->"massless"|>
  },
  "extLegs" -> {
    {bubbleLeg,v1,k1+k2},{treeLeg1,v3,k1},{treeLeg2,v3,k2}
  },
  "loopMomenta" -> {l1},
  "loopExternalMomenta" -> {k1+k2},
  "independentExternalMomenta" -> {k1,k2},
  "ibpMode" -> "full",
  "ispData" -> {},
  "zeroPointRules" -> {
    a0[v1]->alpha1,a0[v2]->alpha2,a0[v3]->alpha3,
    b0[1]->beta1,b0[2]->beta2,b0[3]->beta3
  },
  "symmetryRules" -> {},
  "parityConstraints" -> {
    b[1]+n[1,1]+n[1,2] -> 0,
    b[2]+n[2,1]+n[2,2] -> 0
  }
|>;
```

前两条约束分别选择massive cycle 与massless cycle 的全偶子空间。line 3 的"F" pack 是fixed bridge, 不属于圈积分, 不能出现在parity constraints 中。

先查看提案, 再初始化:

```
proposal = DSKinematics[case];
proposal["selectionTemplate"]

context = DSInit[
  case,
  GenerateDerivativeMetadata -> True,
  ProgressReporting -> True
];
topology = context["topology"];
notation = DSParameterNotation[context];
```

该输入的必要loop 方向只有 k_1+k_2 。它的完整一维Gram 已覆盖bridge 的模长; k_1, k_2 只补齐实际出现的两个独立无圈模长。缺省规则严格为

```
sp[k1+k2,k1+k2] -> ss11^2
sp[k1,k1]        -> sE1^2
sp[k2,k2]        -> sE2^2
```

因此 ss_{11} 是 $|k_1 + k_2|$, 而 sE_1+sE_2 才是 $|k_1| + |k_2|$ 。短名必须与DSParameterNotation 返回的原始sp binding 一起阅读; ss_{ij} 在 $i \neq j$ 时是Gram 点积坐标的根, 不表示 $|k_i + k_j|$ 。

初始化后可以检查公开原子关系和批量 workflow:

```
integral = J[
  {0,0,0},
  {{0,0,0},{0,0,0},{0,0,0}},
  {}
];

timeSeed = dtau[v3,integral,topology];
loopLoopSeed = dq[1,1,integral,topology];
loopExternalSeed = dqk[1,1,integral,topology];
totalDerivative = ds[ss11^2 integral+sE1 integral,sE1,topology];

seeds = DSSeeds[context];
linearData = DSLinear[seeds,context];
```

massive/massless cycle full pack 都是 $\{b, n_1, n_2\}$; fixed bridge full pack 是 $\{F, n_1, n_2\}$, 其中"F" 是数据sentinel, 不是幂指标。bridge 的动量幂进入sector 的结构化prefactor, 不伪造b 指标。解析seed 不要求给数值规则。即使为了缩小Kira 系数域固定dim、nu1、zero-point 或normalization, 所有准备生成DE 的变量 $\{ss_{11}, sE_1, sE_2, E_1, E_2, E_3\}$ 仍必须保持符号; 不得在export 前给它们数值。

本例对应的formal Kira 输入与最终结果分别摘要在06_mix_bubble_tree/kira_input_summary.wl 和06_mix_bubble_tree/kira_result_summary.wl。fresh 输入含178 个seed templates、818217 条canonical relations、818297 条formal equations、310769 个

积分、81 个active masters 和676 个formal targets。WSL Kira 2.3 以w10*1 在160.11s 墙钟时间内完成，unreduced=0；六张DE 均为 81×81 。reduction 的scaling 证书只覆盖记录的exact point，全符号齐次性另由81/81 个source-level Euler+time/q-IBP identities 提供。摘要不作为DSKiraImport 输入；完整工作树已在发布前删除。

2.2 方案一：保留两条外腿，把模长和选作坐标

如果同一顶点的两个外腿指数需要合并成

$$E_0 = |k_1| + |k_2|, \quad (2.2)$$

是否采用 E_0 是用户的运动学参数选择，不由package 猜测。保留原topology 时，先把v3 的相位能量显式改为 E_0 ，再做满秩坐标重定义：

```
boundCase = Join[
  case,
  <|"vertices"->ReplacePart[
    case["vertices"],{3,"externalLegEnergy"}->E0
  ]|>
];
boundContext0 = DSInit[boundCase,GenerateDerivativeMetadata->True];

boundContext = DSRedefineParameters[boundContext0,{
  sp[k1+k2,k1+k2] -> ss11^2,
  sp[k1,k1]       -> (E0-sE2)^2,
  sp[k2,k2]       -> sE2^2
}];
```

数学上这正是 $sE_1 \rightarrow E_0 - sE_2, sE_2 \rightarrow sE_2$ ，但公开API 的规则左端必须写baseCoordinateOrder 中的原始sp[...]，不能直接写 $sE_1 \rightarrow \dots$ 。该坐标图的可求导变量为

```
{ss11,E0,sE2,E1,E2}
```

其中 E_0 只出现一次；ds[expr,E0,boundContext] 同时对显式系数、模长binding 和v3 相位求导。平方规则选择了 $|k_1| = E_0 - sE_2$ 的物理支，使用者应在所研究区域保证 $E_0 > sE_2 > 0$ ；若需要别的分支，必须显式更换参数化。

只改坐标而仍令 $v_3 \rightarrow E_3$ ，不会建立 $E_3 = E_0$ ；相反，只令 $v_3 \rightarrow E_0$ 而不重定义两个模长，也不会告诉package $E_0 = sE_1 + sE_2$ 。这两部分必须同时给出。

2.3 方案二：从输入起采用单一有效外腿

另一种做法是把v3 的两条外腿替换成一条有效外腿。此时topology metadata 已改变，不能把它当作上一个family 的纯坐标改名。为避免同一符号既作矢量又作标量，使用p0 标记有效外腿三动量，用 E_0 标记其模长和顶点相位：

```
singleLegCase = Join[case,<|
  "name" -> "bubbleTreeSingleEffectiveLeg",
  "extLegs" -> {
```

```

    {bubbleLeg,v1,k1+k2},
    {treeLeg0,v3,p0}
  },
  "independentExternalMomenta" -> {p0},
  "vertices" -> ReplacePart[
    case["vertices"],{3,"externalLegEnergy"}->E0
  ]
];

singleLegContext0 = DSInit[
  singleLegCase,
  GenerateDerivativeMetadata->True
];
singleLegContext = DSRedefineParameters[singleLegContext0,{
  sp[k1+k2,k1+k2] -> ss11^2,
  sp[p0,p0] -> E0^2
}];

```

此时 v_3 只有一条外腿，独立变量为 $\{ss11, E_0, E_1, E_2\}$ 。package 只知道 $|p_0| = E_0$ ；“这个 E_0 对应原问题的 $|k_1| + |k_2|$ ”是用户在两个不同topology 之间建立的外部物理对应，不再能从新输入中恢复原来的两个独立模长，也不能分别对它们求导。若后续仍需要 k_1, k_2 的独立变化、交换对称性或软极限，应使用方案一而不是删除两条外腿。

3 SK 费曼规则到积分族

3.1 顶点规则

每个时间顶点 v 有共形时间 $\tau_v < 0$ 和SK 分支 $\sigma_v = \pm$ 。本package 使用

$$\sigma_v = + : \quad e^{-iE_v\tau_v}, \quad \sigma_v = - : \quad e^{+iE_v\tau_v}. \quad (3.1)$$

因此外腿相位对time-IBP 的贡献为

$$\partial_{\tau_v} e^{-iE_v\tau_v} = -iE_v e^{-iE_v\tau_v}, \quad \partial_{\tau_v} e^{+iE_v\tau_v} = +iE_v e^{+iE_v\tau_v}. \quad (3.2)$$

E_v 表示连接该顶点的所有外腿模长在指数中形成的打包能量。它不一定是进入圈内线动量的外部向量。

3.2 内线的四种类型

设内线 e 的有序端点为 $\{u_e, v_e\}$ ，动量为

$$\mathbf{Q}_e = \sum_{\ell=1}^L c_{e\ell} \mathbf{q}_\ell + \sum_{j=1}^K d_{ej} \mathbf{k}_j, \quad q_e = |\mathbf{Q}_e|. \quad (3.3)$$

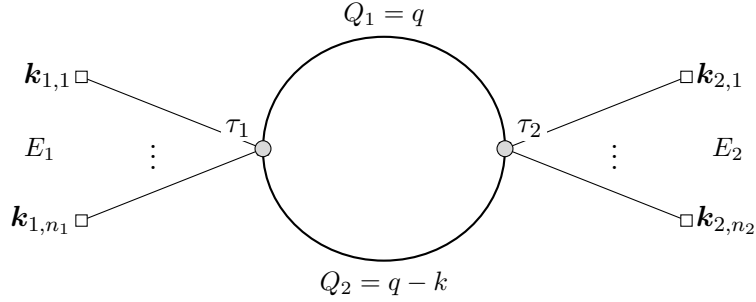
端点的SK 符号决定传播子类型：

质量类型	端点分支	指标包
massive	同分支++ / --	$\{b[e], n[e, 1], n[e, 2]\}$
massive	异分支+- / -+	$\{b[e], n[e, 1], n[e, 2]\}$
massless	同分支++ / --	$\{b[e], n[e, 1], n[e, 2]\}$
massless	异分支+- / -+	$\{b[e], 0, 0\}$

同分支线含theta ordering, 称为full line; 异分支线没有theta 导数产生的shrink, 称为cross line。massive cross 仍有两个Hankel 端点态; massless cross 是单个指数乘积, 因此两个离散槽固定为零。表中给出cycle schema; fixed line 把第一槽 $b[e]$ 换成sentinel "F"。

3.3 Bubble 图作为输入例子

下图只是一个topology 输入例子, 不代表生成器硬编码bubble。两条内线可分别选为massive/massless full/cross; 只需改变family metadata, 统一积分Head 不变。



例如mixed bubble 可取 $Q_1 = q$ 为massive, $Q_2 = q - k$ 为massless。pure massless bubble 则把两线都设为massless。顶点符号从++ 改成+- 时, 两条线自动从full 变为cross; 不是换一套积分Head。

4 按mode 固定的三参数积分表示

所有top/sub-sector 都使用三个正式槽位

$$J[\{a_v\}, \{\text{pack}_e\}, \{n_{\text{isp}, r}\}] \equiv J[\text{aList}, \text{linePacks}, \text{ispList}]. \quad (4.1)$$

三个槽位分别是

1. **aList**: 当前sector 的独立时间变量幂次;
2. **linePacks**: 按固定line slot 排列的传播子/缩并线指标包;
3. **ispList**: ISP 的正式指标槽, 不是附注metadata。

式 (4.1) 是full 模式的schema。time-only 公开schema 为

$$J[\text{sectorKey}, \text{timeShifts}, \text{stateBits}]. \quad (4.2)$$

sectorKey 是root propagator 顺序的定长字符串；**timeShifts** 按当前sector 的compact vertex components 排列；**stateBits** 是 n_i 类离散函数态，不是第二套sector key。初始化一旦选择mode，公开入口就只接受对应schema。

实际幂次包含初始化零点：

$$A_v = a_v + a0_v, \quad B_e = b_e + b0_e, \quad B_e^S = bS_e + bS0_e. \quad (4.3)$$

质量、 ν_e 、端点、SK 符号、动量表达式、顶点能量、外不变量名称和symmetry rules 都不写入 J 。

theta 导数缩并线时，pack 变为

$$\text{pack}_e = \{bS_e\}. \quad (4.4)$$

delta 函数合并两个时间变量，所以**aList** 只保留一个代表顶点槽；原顶点到compact slot 的映射保存在**sectorMetadataList**。line slot 始终保留，不能删除 bS_e 。

5 Massive 导数、EOM 与缩并

5.1 端点导数与即时EOM

massive full/cross 的两个端点态均取 $n_{e,r} = 0, 1$ 。time 导数先产生

$$\partial_{\tau_r} : \quad -J[\dots, \{b_e - 1, n_{e,r} + 1, \dots\}, \dots]. \quad (5.1)$$

若 $n_{e,r} = 1$ ，二阶态必须立刻EOM。对h：

$$J[n_{e,r} = 2] = -J[n_{e,r} = 0] - (2\nu_e + 1)J[a_r - 1, b_e + 1, n_{e,r} = 1]. \quad (5.2)$$

对裸H：

$$J_H[n_{e,r} = 2] = -J_H[n_{e,r} = 0] - J_H[a_r - 1, b_e + 1, n_{e,r} = 1] + \nu_e^2 J_H[a_r - 2, b_e + 2, n_{e,r} = 0]. \quad (5.3)$$

另一端点指标不变。最后一项就是 ν_e^2/x^2 二次pole；loop-momentum 导数若产生 $n = 2$ ，同样在seed 层立即使用对应EOM。当前实现由统一函数系统编译这些递推；旧版内置公式只作为历史基线。

5.2 通用 2×2 函数系统输入

当前实现对每条massive line 输入

$$f'' + P(x)f' + Q(x)f = 0, \quad \mathbf{F} = T(x)(f, f')^T, \quad x = -\xi\tau, \quad (5.4)$$

其中 P 是一阶导系数、 Q 是零阶系数， T 缺省为单位矩阵；还必须给原始导数基底的完整Wronskian W 。初始化层统一生成

$$A_0 = \begin{pmatrix} 0 & 1 \\ -Q & -P \end{pmatrix}, \quad A_T = T'T^{-1} + TA_0T^{-1}, \quad W_T = \det(T)W. \quad (5.5)$$

IBP 的time/radial 导数只调用由 A_T 编译的移位数据，theta shrink 只调用由 W_T 编译的数据；不在IBP 层重算 P, Q, T, W 。缺省massive 模式是h，直接取 $P_h = (2\nu + 1)/x$ 、 $Q_h = 1$ 、 $T = I_2$ 与完整 W_h ，所以 $W_T = W_h$ 。裸H 是另一个 $T = I_2$ preset。若从H 输入经一般基变换构造h，则

$$\mathbf{h} = x^{-\nu} \begin{pmatrix} 1 & 0 \\ -\nu/x & 1 \end{pmatrix} \mathbf{H}, \quad W_h = x^{-2\nu} W_H. \quad (5.6)$$

该编译接口已实现；显式 W_T 只作 $W_T = \det(T)W$ 的一致性校验。缺省h、裸H、一般 T 和非法输入由函数系统专项检查覆盖，h/H 另有独立物理benchmark。Wronskian 的标准推导见技术笔记专门subsection。

用户输入一般函数系统时，在对应massive line 中加入

```
"functionSystem" -> <|
  "variable" -> x,
  "P" -> Pexpr,
  "Q" -> Qexpr,
  "T" -> {{t11,t12},{t21,t22}},
  "W" -> Wexpr,
  "WT" -> Automatic,
  "shrinkBShift" -> 1,
  "shrinkZeroPointShift" -> zeroShift
|>
```

也可在放入line 前调用

```
compiled = compileFunctionSystem[functionSystem];
compiled["status"]
compiled["AT"]
compiled["WT"]
```

检查编译结果。T 省略时为单位矩阵；输入必须通过可逆性、Abel/Wronskian 一致性和有限Laurent 编译门禁。shrinkBShift 与shrinkZeroPointShift 是由函数系统和Wronskian 决定的物理编译数据，不是普通显示或basis 选项。使用内建h/H preset 时应保留缺省值；一般函数系统也应优先让编译结果决定这两个量。

5.3 Massive theta shrink

只有massiveFull 且 $n_{e,1} + n_{e,2} = 1$ 时有Wronskian shrink。端点 r 的系数为

$$-\chi_e (-1)^{n_{e,r}} \mathcal{C}_e, \quad \chi_e = \begin{cases} +1, & ++, \\ -1, & --, \end{cases} \quad \mathcal{C}_e = \frac{4i}{\pi} e^{\pi \text{Im} \nu_e}. \quad (5.7)$$

其中 χ_e 只由两个端点的vertexType 派生，用户不能覆盖。child sector 的normalization 吸收 $\chi_e \mathcal{C}_e$ ；因此normalized contact 中contour sign 与Wronskian 常数各消去一次，但有序Wronskian 的整体负号保留；不会在raw contact 之后重复乘branch sign。

对h 模式：

$$a_{\text{merged}} = a_u + a_v - 1, \quad a0_{\text{merged}} = a0_u + a0_v - 2\nu_e, \quad (5.8)$$

$$bS_e = b_e + 1, \quad bS0_e = b0_e + 2\nu_e. \quad (5.9)$$

对 H 模式整数移位相同，但两个 $2\nu_e$ zero-point shift 都为零。massiveCross 没有theta shrink。

因此现行缺省值及其实际child 映射为

模式	(s, z)	a_{merged}	$a0_{\text{merged}}$	bS_e	$bS0_e$
h	$(1, 2\nu_e)$	$a_u + a_v - 1$	$a0_u + a0_v - 2\nu_e$	$b_e + 1$	$b0_e + 2\nu_e$
H	$(1, 0)$	$a_u + a_v - 1$	$a0_u + a0_v$	$b_e + 1$	$b0_e$

这里 s 进入整数指标， z 进入zero point。对带圈动量的line，contact 支持 $n_{e,1} + n_{e,2} = 1$ ，故两种缺省模式都满足

$$b_e + n_{e,1} + n_{e,2} \equiv b_e + 1 = bS_e \pmod{2}. \quad (5.10)$$

所以缺省h/H shrink 不改变subsector parity。h 的 $2\nu_e$ 是zero point 而非整数seed 指标，H 的zero-point shift 为零；fixed/non-loop line 不进入parity 判定。对fixed/non-loop line 不保留 b/bS 指标槽，缺省 z 进入target sector 的结构化prefactor，normalized contact coefficient 保留整数部分对应的模长幂 r^{-s} 。

一般不建议用户修改shrinkBShift 或shrinkZeroPointShift。若物理函数系统确实要求override，必须同时重新建立child-sector zero point、跨sector normalization coefficient 和parity metadata；只改一个shift 而沿用其它缺省数据会得到不一致的reduction/DE。若相对缺省的zero-point 重基不能精确判定为整数，相应sector 的parity 功能应关闭。需要额外吸收显式模长幂时，应通过后续master/basis 选择处理，而不是另改shrink shift。

6 Massless 双端点quotient 与time-IBP

6.1 有序端点定义

对有序端点 $\{u, v\}$ ，令 $\Delta = \tau_u - \tau_v$ 。masslessFull 的同分支符号记为 $\sigma = +1$ 对 $++$ 、 $\sigma = -1$ 对 $--$ ：

$$M_0 = \theta(\Delta)e^{-i\sigma q\Delta} + \theta(-\Delta)e^{+i\sigma q\Delta}, \quad (6.1)$$

$$M_1 = -\theta(\Delta)e^{-i\sigma q\Delta} + \theta(-\Delta)e^{+i\sigma q\Delta}. \quad (6.2)$$

第一端点定义 M_1 的方向。交换端点时 M_0 不变、 M_1 变号。公开双端点态记为 $F_{n_1 n_2}$ ，并用

$$F_{00} = M_0, \quad F_{10} = M_1, \quad F_{01} = -F_{10}, \quad F_{11} = -F_{00} \quad (6.3)$$

定义quotient。正式 J 始终保存三槽 $\{b_e, n_{e,1}, n_{e,2}\}$ ，两个端点指标均只取0,1；relation 层以 $n_{e,2} \rightarrow 0$ 为canonical 方向，不建立massless $n = 2$ 状态。

这里的 $F_{11} = -F_{00}$ 已抽出每次端点导数的 $\pm i\sigma q$ 。原始时间导数关系仍为 $\{11\} = +q^2\{00\}$ ：cycle line 由两次 $b \rightarrow b - 1$ 承载 q^2 ，fixed/独立外动量line 由导数算符显式乘两次模长参数。用户不需要为同一代数类重复生成source seeds；DSSeeds 只生成00/10，但导数产生的01/11 会连同这些系数立即约回。返回的seedTemplateSummary["discreteStateAudits"] 记录raw 四态数、代表态数、删除数和canonical direction。

每条massless line 仍保留自己的三槽pack；只有共享同一当前代表顶点对的time boundary 按共同theta 组合。fixed line 对应写成 $\{\text{"F"}, n_{e,1}, n_{e,2}\}$ 。

6.2 两个端点的regular 与delta 规则

由式 (6.2),

$$\partial_{\tau_u} M_n = +i\sigma q M_{1-n} - 2n \delta(\tau_u - \tau_v), \quad (6.4)$$

$$\partial_{\tau_v} M_n = -i\sigma q M_{1-n} + 2n \delta(\tau_u - \tau_v). \quad (6.5)$$

对一般公开状态先应用式 (6.3)。在canonical representative $\{b, n, 0\}$ 上, 四个基本端点检查为

n	端点	regular 指标变化与系数
0	第一端点	$+i\sigma \{b-1, 1, 0\}$
0	第二端点	$-i\sigma \{b-1, 1, 0\}$
1	第一端点	$+i\sigma \{b-1, 0, 0\}$
1	第二端点	$-i\sigma \{b-1, 0, 0\}$

同一端点连续作用两次, regular 部分回到原canonical representative:

$$\partial_{\tau_u}^2 : -J[\{b-2, n, 0\}], \quad \partial_{\tau_v}^2 : -J[\{b-2, n, 0\}]. \quad (6.6)$$

6.3 Massless shrink 与抵消

公开奇数端点态 $n_1 + n_2 = 1$ 才产生theta-delta; 在canonical representative 中即 $\{n_1, n_2\} = \{1, 0\}$:

$$\partial_{\tau_u} : -2\delta(\tau_u - \tau_v), \quad \partial_{\tau_v} : +2\delta(\tau_u - \tau_v). \quad (6.7)$$

massless shrink 没有Hankel Wronskian prefactor, 也没有 2ν shift:

$$a_{\text{merged}} = a_u + a_v, \quad a0_{\text{merged}} = a0_u + a0_v, \quad bS_e = b_e, \quad bS0_e = b0_e. \quad (6.8)$$

这里缺省compiled shift 是 $(s, z) = (0, 0)$ 。“没有shift”的准确含义是shrink 不新增任何整数幂或zero-point 幂吸收。对cycle line, 式 (6.8) 直接保留 $bS_e = b_e$ 、 $bS0_e = b0_e$ 。对fixed/non-loop line, source sector 已有的模长幂必须保存在完整 N_s ; target 收缩后按其自身 N_t 归一化。contact 一律先生成裸核系数 c_{raw} , 再写成

$$c_{\text{raw}} \frac{N_s}{N_t} J_t. \quad (6.9)$$

因此atomic fixed massless ++ 的source 与target 都含 $sE1^{-\beta_1}$, 比值为1, normalized contact 系数精确为-2。删掉target 中原有模长幂会错误得到 $-2/sE1^{\beta_1}$ 。同一sectorPrefactorData 同时服务contact、ds/DSDE 与反向被积函数恢复, 禁止在任一consumer 另猜一次模长幂。

massless 的subsector parity 不保持原offset。双端点表示中contact 支持满足 $n_1 + n_2 = 1$, 而 $bS = b$, 所以若parent 使用 $b + n_1 + n_2$ 分级,

$$b + n_1 + n_2 = b + 1 = bS + 1 \pmod{2}. \quad (6.10)$$

因此cycle massless contact 使child-sector parity offset 相对parent 翻转1; fixed/non-loop massless line 不参与parity, 没有这次翻转。多线simultaneous contact 按

被shrink 的massless cycle line 数模2 累加。package 不自动给pure massless family 启用parity; mixed h/H 系统一旦使用已认证root parity, massless cycle transition 必须携带上述affine offset, 不能套用h/H 的“缺省不变”。

缩并后, 剩余传播子的两个原端点可能映到同一个active vertex。此时:

1. time 导数必须同时作用于两个端点, 不能只取第一个匹配;
2. 两个regular 端点贡献按式 (6.5) 抵消;
3. 两个theta-delta 贡献的 $-2/ + 2$ 系数相消, 不再产生shrink 项;
4. 反对称态 $n = 1$ 在coincident endpoint 恒为零。

这个零关系也必须作用于从top 方程产生的目标sub-sector 项。当前实现根据每个 J 中已经变成单元素pack 的shrunk lines, 重建目标sector 的顶点代表映射, 再判定coincident $n = 1$; 不能只读取source top 的endpoints。

对 $+-$ 或 $-+$ masslessCross, 没有 n 、theta 或shrink。端点regular 系数由该端点分支决定:

$$+\text{端点}: +i\{b-1\}, \quad -\text{端点}: -i\{b-1\}. \quad (6.11)$$

6.4 同一顶点对多条full lines

同一当前代表顶点对的massive/massless full lines 共享时间差 Δ 。写

$$G_e = \theta(\Delta)A_e + \theta(-\Delta)B_e, \quad (6.12)$$

则整个bundle 的boundary 只有

$$\delta(\Delta) \left(\prod_e A_e - \prod_e B_e \right). \quad (6.13)$$

定义 $J_e = (A_e + B_e)/2$ 、 $D_e = A_e - B_e$, 可保持现有逐线pack:

$$\prod_e A_e - \prod_e B_e = \sum_{\substack{\emptyset \neq S \subseteq B \\ |S| \text{ odd}}} 2^{1-|S|} \prod_{e \in S} D_e \prod_{e \notin S} J_e. \quad (6.14)$$

一次事件可同时shrink 任意非空奇数条bundle 线, 系数为 $2^{1-|S|}$, 但只合并一次顶点且只含一个delta。两线只有single contacts; 三线另有系数1/4 的triple contact。未选择的coincident full lines 立即应用massless $n = 1$ 为零和massive endpoint-swap canonical, 不会再次产生theta boundary。

sector 只允许连接两个不同当前代表类的contact 事件, 按BFS 枚举; 事件之间形成forest, 但单个事件可选择多条平行线。单传播子的对称equal-time 值取 $\theta(0) = 1/2$, 但不把它作为 $\delta\theta^m$ 的点值乘法规则。保留逐线theta 时, 统一Gaussian mollifier 给出的线别积分在求和后严格等价于式 (6.14)。

例如三条同向masslessFull 线均取 $n_1 = n_2 = n_3 = 1$, 并从第一有序端点求导。此时每条线的对称等时因子为零、contact difference 为 -2 , 所以三个single-contact 项都因未选中线的等时因子为零而消失, 只剩

$$\frac{1}{4}(-2)^3 = -2. \quad (6.15)$$

指标上即

$$J[\{a_u, a_v\}, \{\{b_1, 1, 0\}, \{b_2, 1, 0\}, \{b_3, 1, 0\}\}, \{\cdots\}] \rightarrow -2J[\{a_u + a_v\}, \{\{b_1\}, \{b_2\}, \{b_3\}\}, \{\cdots\}]. \quad (6.16)$$

三个pack 同时shrink, 但只合并一次顶点且没有额外massless 幂次shift。共同-theta与Gaussian 逐线方案的完整推导、massive W_T 例子和等价条件见技术笔记附录。

7 Topology、标量积、ISP 与外腿能量

7.1 输入骨架

```
case = <|
  "name" -> "mixedBubble",
  "vertices" -> {
    <|"id"->v1,"vertexType"->"+",
      "externalLegEnergy"->Sqrt[sp[kE,kE]]|>,
    <|"id"->v2,"vertexType"->"+", "externalLegEnergy"->E2|>
  },
  "lines" -> {
    <|"id"->1,"endpoints"->{v1,v2},"momentum"->q,
      "nu"->nuM,"massType"->"massive"|>,
    <|"id"->2,"endpoints"->{v1,v2},"momentum"->q-k,
      "massType"->"massless"|>
  },
  "loopMomenta" -> {q},
  "loopExternalMomenta" -> {k},
  "independentExternalMomenta" -> {kE},
  "ibpMode" -> "full",
  "ispData" -> {},
  "zeroPointRules" -> {
    a0[v1]->alpha1,a0[v2]->alpha2,
    b0[1]->beta1,b0[2]->beta2
  },
  "symmetryRules" -> {}
|>;
```

`endpoints` 对masslessFull 是有序输入；交换顺序会改变 $n = 1$ 表示的符号。

输入后推荐立即通过公开初始化入口运行

```
context = DSInit[case, GenerateDerivativeMetadata -> True];
topo = context["topology"];
validation = context["validation"];
derivatives = context["derivatives"];
```

`validation["issues"]` 给出topology/ISP/函数系统问题；初始化metadata 同时保存sector、convention 和可选微分算符信息。

7.2 sp 与外不变量

用户端所有圈动量相关标量积统一写 $\text{sp}[p, r]$ 。代码直接使用

```
SetAttributes[sp, Orderless];
```

实现 $\text{sp}(p, r) = \text{sp}(r, p)$ 。这只是标量积交换性，不是图或积分族对称性。

用户可给圈动量和外动量任意符号名，例如`sah, bob, alice`。package 不从符号字符串猜测角色，也不再从一个统一列表自动分类；输入必须显式声明

```
loopMomenta
loopExternalMomenta
independentExternalMomenta
```

`loopExternalMomenta` 是圈积分外动量基，决定完整Gram 坐标、 dqk 生成元和ISP 闭合维数；`independentExternalMomenta` 是topology 中实际无圈line/phase 的独立模长列表，只定义 $|P_e|$ ，不定义彼此点积。package 不读取旧`externalMomenta/externalLegMomenta`；程序会从line routing 独立计算实际需要的空间并与用户声明比较，而不会替用户选择基。

本手册以后把`loopExternalMomenta` 的第 i 项简称为 kL_i ，把`independentExternalMomenta` 的第 e 项简称为 kE_e 。这是按两个输入列表的位置定义的角色名，不要求用户真的把动量符号写成 kL_1/kE_1 ；`alice`、`bob` 等任意符号同样合法。两套编号各自从1 开始且互不共享： kL 第 i, j 项在内部形成 $qk[*, i]$ 、 $kk[i, j]$ ，公开为 $ssij$ ； kE 第 e 项的模长平方在坐标/Jacobian 层使用`externalLegSquaredCoordinate[e]`，公开为 sEe ，不进入loop-IBP/ISP 的 K 。

结构圈数由内部无向多重图的第一Betti 数

$$L = E - V + C \quad (7.1)$$

给出；平行边逐条计数，自环各贡献一圈，`extLegs` 不计入 E 。逐边删除后连通分量增加者为bridge，其余为cycle line。`ibpMode->"full"` 还要求`loopMomenta` 数量等于 L 、routing 满秩并位于incidence cycle space；`ibpMode->"timeOnly"` 不生成momentum IBP，树图缺省即为该模式。

为消除任意圈动量平移，程序把line routing 写成 $Q = Aq + r$ ，选满秩参考行 A_R 后使用

$$q' = A_R q + r_R, \quad r' = r - A A_R^{-1} r_R. \quad (7.2)$$

因此 $\{q+k_1, q+k_1+k_2\}$ 实际只要求方向 k_2 ；加减号始终保留精确系数。这个计算只用于审计用户列表是否完备，不用于自动改写用户选定的动量基。

内部保留用户给定loop-external 基的平方Gram 原子，公开缺省为根号坐标：

```
kk[i, j] = sp[ki, kj]
ssij = Sqrt[sp[ki, kj]]
```

用户输出不保留内部 $kk[i, j]$ 。

7.3 ISP 完备性

L 圈、 K 个独立外动量共有

$$N_{\text{sp}} = \frac{L(L+1)}{2} + LK \quad (7.3)$$

个圈相关独立标量积。用户定义的propagator 模长平方和ISP 必须构成可反解坐标。ISP 可以是一般标量积，例如

`sp[13,k321+13]`

而不只限于 $q_i \cdot q_j$ 。package 验证闭合性，但不自动删线或替用户重选propagator basis。

每个ISP 使用name/expr/range Association，或等价三元组{name,expr,range}:

```
"ispData" -> {
  <|"name"->rho1,"expr"->sp[q1,k],"range"->{0,1}|>,
  <|"name"->rho2,"expr"->sp[q2,k],"range"->{0,1}|>
}
```

name 必须唯一，expr 使用已声明的动量基，range 给出整数指标seed 值域。ISP 的定义零点固定为0: 正值是numerator 幂；用户显式给负值时表示该标量积坐标的额外denominator，package 不阻断。用户不需要也不能用输入改变这一零点。独立benchmark 与package 使用同一schema。

7.4 实际无圈模长与fixed line

independentExternalMomenta 中每个表达式定义一个实际无圈模长。程序要求它们覆盖所有active 无圈line/被审计phase 中不能由完整loop Gram 表示的模长；不会从向量原子表自动拆分或补齐。

程序把实际无圈模长平方在完整loop Gram 与此前已选无圈平方坐标上按首次出现顺序做秩筛选，只为独立项建立模长变量：

```
sE1 = Sqrt[sp[PE1,PE1]], sE2 = Sqrt[sp[PE2,PE2]], ...
```

未入基的实际出现项保存为当前平方坐标的dependent binding；这些模长不进入loop IBP generator 或ISP closure。与两组动量都无关、只进入顶点相位的能量仍使用独立标量 $ke[i]$ 。

例如

```
"loopExternalMomenta" -> {k1,k2}
"independentExternalMomenta" -> {kE1,kE2,kE1+kE2}
"lines" -> {...,"momentum"->kE1,...}
"vertices" -> {<|"id"->v1,"vertexType"->"+" ,
  "externalLegEnergy"->Sqrt[sp[kE1+kE2,kE1+kE2]]|>,...}
(* loop Gram: {ss11,ss12,ss22}; no-loop magnitudes: {sE1,sE2,sE3} *)
```

无圈模长只把整体反号 $P \leftrightarrow -P$ 视为同一对象；|alice+bob| 与|alice-bob| 必须分别声明，不能合并。

坐标名只由两个用户列表各自的稳定顺序决定，与符号名无关。每套列表的总数为1-9 时使用ss11/sE1；10-99 时使用两位补零，例如ss0101,ss0110,sE01；100 起按该列表总数位数自然扩展。用户若需要其它坐标，统一通过KinematicRules 或DSRedefineParameters 给出：

DSInit[case, KinematicRules->{sp[k1,k1]->u^2,...}]

特别地，若topology 实际分别出现 $|k_{E1}|$ 、 $|k_{E1} + k_{E2}|$ 、 $|k_{E2}|$ ，用户就应在independentExternalMomenta 中给出这三个表达式，缺省得到sE1,sE2,sE3；程序既不输出sp[kE1,kE2]，也不用余弦定理消去中间变量。若再声明 $2k_{E1}$ ，它是过完备项并触发warning。

full 模式的linePacks 始终按root lineOrder 保留每条传播子的位置。cycle/fixed full pack 分别为{b,n1,n2} 与{"F",n1,n2}；收缩后分别为{bS} 与{"F"}。因此fixed line 没有b/bS，但模长幂仍属于该sector 的结构化prefactor。time-only 的这些packs 只在Private producer 内存在，由中央adapter 映射到式 (4.2)。

每个sector 的完整normalization 写为结构化 N_s 。无圈模长部分按初始化时independentExternalMomenta 的稳定编号保存为kEpower[be1,be2,...]；当前参数表达式单独保存在kEParameterExpressions。例如用户重新定义参数后，一个编号模长可以映成若干新变量的组合，但幂向量和line/sector 身份不变；ds 根据表达式映射做链式法则，不能把kEpower 当成符号名猜测或只取一个单项式。已收缩massive line 的Wronskian normalization 按line index 保存于contractionParts，simultaneous contact 对每条线只累计一次。

初始化采用不阻塞脚本的两阶段接口：

```
proposal = DSKinematics[case];
(* 查看 proposal["defaultRules"] 与 proposal["selectionTemplate"] *)
custom = {sp[k1,k1]->u^2, sp[k1,k2]->u v,
          sp[k2,k2]->v^2+w^2, ...};
audit = DSKinematics[case, custom];
context = DSInit[case, KinematicRules->custom];
```

DSKinematics 的返回值分组列出：结构图/routing 审计、两个用户列表的exact/over/under 状态、基础坐标顺序、动力学规则矩阵与参数Jacobian、零空间和capability。主要Association key 包括

```
momentumDeclarationStatus, kinematicCoordinateStatus,
dependentMagnitudeBindings, missingDirections, missingMagnitudeSquares,
capabilities.
```

欠完备动量列表或动力学规则会以红色error 显示缺失方向/零空间并令DSInit 返回失败；所有后续模块读取同一capability，不能继续生成seed。过完备以红色warning 提醒，允许初始化和symbolic IBP seed，但关闭ds/DSDE 与唯一rep2innerform。对于过完备loop 声明，用户原列表保留作审计，实际 n_K 、Gram、dqk 与ISP 闭合只使用affine quotient 必要基effectiveLoopExternalMomenta；共同loop shift 不会虚增标量积空间。只有exact 状态同时开放初始化、time/momentum IBP、求导和反变换。

成功初始化后可直接查看或重定义notation：

```
notation = DSParameterNotation[context];
newContext = DSRedefineParameters[context, {
  sp[k,k] -> loopScale^2,
  sp[p1,p1] -> leg1^2,
  sp[p2,p2] -> leg2^2,
```

```
sp[p1+p2,p1+p2] -> leg12^2
}];
```

新规则必须完备；返回的新context 有新的审计和inputHash，后续seed、ds 与DSDE 都读取它。规则右端可以是一般混合参数化，例如 $(u+v)^2$ ；程序先展开每条右端并提取原子参数 u, v ，再建立完整Jacobian。参数数多于基础坐标数时属于过完备并关闭求导。正式函数名是DSRedefineParameters。

7.5 独立变量与微分方程求导

当前版本提供seed 层和表达式层的独立变量求导接口。变量分三类：

- 圈线相关外动量模长，缺省为ssij，通过外动量矢量导数组合实现。
- 实际无圈动量模长独立基，缺省为sE1,sE2,...；从属模长先写成这些变量与ssij 的binding。若它对应一条传播子，导数按binding Jacobian 同时作用于该线的分母和massive/massless building block。
- 独立顶点能量参数，例如ke[i]；它不是动量向量，不定义点积。

对顶点能量 y ,

$$\frac{\partial}{\partial y} J = \sum_v \left[-\eta_v \frac{\partial E_v}{\partial y} \right] J[a_v \rightarrow a_v + 1], \quad \eta_v = \begin{cases} -i, & v = +, \\ +i, & v = -, \end{cases} \quad (7.4)$$

其中 $E_v = \text{vertexExternalEnergy}[\text{topo}, v]$ 。额外负号来自本package 的时间幂convention $(-\tau_v)^{a_v+a_{0v}}$ ：相位对 E_v 求导产生 $\eta_v \tau_v = -\eta_v (-\tau_v)$ 。

对外不变量 x_a ，先引入外动量矢量导数

$$D_{ij} = k_i \cdot \frac{\partial}{\partial k_j}, \quad D_{ij}(k_m \cdot k_n) = \delta_{jm}(k_i \cdot k_n) + \delta_{jn}(k_i \cdot k_m). \quad (7.5)$$

然后在选定外不变量坐标 $\{x_b\}$ 上解约束方程

$$\frac{\partial}{\partial x_a} = \sum_{ij} c_{ij}^{(a)} D_{ij}, \quad \sum_{ij} c_{ij}^{(a)} D_{ij} x_b = \delta_{ab}. \quad (7.6)$$

完整 K^2 个 D_{ij} 通常含零空间，解不唯一；当前实现默认使用上三角external-vector basis 取一组canonical 解，并在decomposition 数据中显式返回矩阵、系数、残差、nullity 与nonUniqueQ。

相关接口为

```
makeExternalInvariantDerivativeDecomposition[topo, var]
applyExternalVectorDerivativeSeed[topo, int, gen]
applyExternalInvariantVariableDerivativeSeed[topo, int, var]
applyIndependentVariableDerivativeSeed[topo, int, var]
ds[expr, ssij, context] (* context["topology"] 也可 *)
ds[expr, ssij]
```

`applyIndependentVariableDerivativeSeed` 会自动分派：loop Gram 变量先做上述decomposition，再把每个 D_{ij} 作用到传播子、building block、ISP 和相位；实际无圈模长及其从属binding 使用line-local 径向导数并微分相位；其它变量按独立顶点能量标量处理。一般用户坐标 u 直接使用 $\sum_a (\partial x_a / \partial u) \partial_{x_a}$ ，不会因 u 同时进入多个Gram 原子或binding 而提前返回。

`ds` 是表达式级总导数入口。变量必须使用函数族初始化后的外部名字；内部`kk[i,j]` 不接受。若

$$\mathcal{I}(s) = \sum_r c_r(s) J_r(s) + c_0(s), \quad (7.7)$$

则

$$\partial_s \mathcal{I} = \sum_r c'_r(s) J_r(s) + \sum_r c_r(s) \partial_s J_r + c'_0(s). \quad (7.8)$$

程序先将 J_r 替换为惰性token 求显式系数导数，再补上每个积分的指标导数，合并后统一执行EOM、symmetry 与canonical。它接受单个 J 和任意线性组合，拒绝 $J_i J_j$ 等非线性输入。

令 $x_{ij} = \text{sp}(k_i, k_j) = \text{ssij}^2$ 。package 不重写平方Gram 导数，而是使用 $\partial_{\text{ssij}} = 2 \text{ssij} \partial_{x_{ij}}$ 调用原子层。显式系数与sector prefactor 仍执行完整乘积与链式法则；程序不使用PowerExpand。

对无圈massive 线 e ，记 $B_e = b_e + b_0 e$ 。径向导数包含 $-B_e J[b_e \rightarrow b_e + 1]$ ；若 A_T 的某个Laurent 项为 $c x^p$ 并把端点态 α 送到 β ，则另有

$$c J[a_v \rightarrow a_v + p + 1, b_e \rightarrow b_e - p, n_{e,v} : \alpha \rightarrow \beta]. \quad (7.9)$$

这直接读取与IBP 相同的`derivativeTerms`，不读取只属于time-theta shrink 的`WT/shrinkTerms`。

发布的examples 分两级：single-massive sunrise 只比较general IBP seeds 与general 参数微分算符；只有pure massive bubble 与mix bubble+tree 进入外部reduction 流程，其中massive bubble 继续承担reference/dlog/DE/scaling 对照。

对massive building block, external-vector/外不变量导数与qIBP、tIBP 自动使用同一份line-local `compiledFunctionSystem`。普通导数只读取最终 A_T 编译的`derivativeTerms`，因此用户选择 h 、裸 H 或一般 P, Q, T, W 后不需要为微分方程另设convention。 $W_T = \det(T)W$ 及其`shrinkTerms` 只用于time-IBP 的theta coincidence/Wronskian shrink；普通动力学量导数不产生theta shrink，因而不调用 W_T 。

7.6 用户对称性规则

一般积分族对称性依赖质量和外参条件，当前实现不自动猜测一般图automorphism。程序会自动加入受门禁保护的原始self-loop tadpole 规则，并与用户`symmetryRules` 取去重并集：

```
repSymmetry0[topo_]
effectiveSymmetryRules0[topo_]
symmetry[expr_, topo_]
```

`repSymmetry0` 只返回用户原始规则；`symmetry` 对并集只做一次替换。自动规则只识别原始端点相同的self-loop，不作用于cross propagator 或shrink 后才coincident 的普通线；odd ISP 归零还要求对应loop momentum 不被其它传播子共享。

8 IBP 生成元与公开接口状态

8.1 完整生成元

每个active vertex 有一个time generator。对 L 个圈动量和 K 个loopExternal Momenta, loop-momentum generators 为

$$\mathcal{O}_{\ell,v} = \frac{\partial}{\partial q_{\ell}^{\mu}} v^{\mu}, \quad v \in \{q_1, \dots, q_L, k_1, \dots, k_K\}, \quad (8.1)$$

总数 $L(L + K)$ 。不能漏掉 $d/dq_{\ell} \cdot q_m$ 的交叉项或 $d/dq_{\ell} \cdot k_j$ 。

当前核心使用applyTimeGeneratorSeed 与applyMomentumGeneratorSeed。微分方程方向的外部变量求导使用上一节列出的applyIndependentVariableDerivativeSeed 等接口。自动batch 必须在给定离散 n 后调用, 再立即EOM/canonical。

8.2 公开原子API

package 保留以下原子接口。推荐显式传入DSInit context 或parsed topology; 若先调用setIBPTopologyContext[topo], 也可使用短签名:

```
dtau[i,expr,context]      or dtau[i,expr]
dqq[i,j,expr,context]    or dqq[i,j,expr]
dqk[i,j,expr,context]    or dqk[i,j,expr]
ds[expr,var,context]     or ds[expr,var]
rep2innerform[expr,context]
rep2outform[expr,context]
rep2Integrand[expr,context]
```

上式第三参数也可写成context["topology"]。resolver 会先识别完整context 并解包, 不会把它误作原始topology case 再次解析。前三个复合算子分别表示 $d/d\tau_i$ 、 $d/dq_i \cdot q_j$ 、 $d/dq_i \cdot k_j$, 对 J 的线性组合保持线性, 并复用现有EOM、theta boundary 与canonical。

例如先把离散态替换为字面0/1, 再调用

```
int = makeBaseIntegral[topo] /. seedRules;
timeRelation = dtau[v1,int,topo];
qqRelation = dqq[1,1,int,topo];
qkRelation = dqk[1,1,int,topo];

setIBPTopologyContext[topo];
sameTimeRelation = dtau[v1,int];
clearIBPTopologyContext[];
```

短签名依赖唯一的已注册topology; 同时处理多个topology 时应始终使用显式参数。

rep2innerform/rep2outform 转换用户动量与内部编号、sp 与qq/qk/kk、外不变量名和ISP 名。实现会先保护所有 J , 因此不改变三个指标槽、line pack 次序或任何 $a/b/n/ispN$ 值。

rep2Integrand 把 J 展开为被积函数。普通时间幂、顶点相位、分母幂和ISP 直接相乘; 每条未缩并传播子的非平凡Hankel/指数/theta block 用惰性Hh[...] 包装。它不积分、不生成IBP、不应用EOM。

9 Seed、linearData 与serializer

canonical seed 必须满足:

- 每个实际sector 都有全部active time 和 $L(L + K)$ momentum generators;
- massive 无 $n \geq 2$;
- masslessFull 只有 $n = 0, 1$;
- 共同-theta odd-subset contact、contact 可达sector 与coincident canonical 已处理;
- 解析seed 保留非零 $a_0/b_0/b_{S0}$ 。

9.1 完整离散模板与连续指标撒点

DSSeeds 的"allSeeds" 是稳定的一维模板列表; 也可以用DSAllSeeds[seedData] 读取, 或用无参DSAllSeeds[] 读取最近一次成功结果。"seedGroups" 按缺省的{sectorKey, ibpClass, generator} 分组, 可由DSSeedGroups 读取; DSSeedGroupMetadata 返回同序来源说明。内部多重Table 在公开前用Flatten[... , Infinity] 磨平。每个模板已经对该sector 的massive 端点遍历0,1, masslessFull 则完整遍历两个quotient representatives 00/10, 随后完成EOM/canonical; 连续 a_i, b_i, b_{S_i} 和ISP 指标仍保持general。该代表态缩减是已证明的代数canonical, 不是离散抽样, 也不是parity 筛选。连续指标交给DSMetaSeedRange/DSGenerateIBP 统一撒点。若EOM/canonical 把某个完整离散态化为精确零, 该密封模板仍作为合法恒等式保留在计数、哈希和离散态完整性审计中; 接口只拒绝缺失equation, 或非零且完全不含J 的伪模板。

先初始化逐组shift metadata:

```
groups = DSSeedGroups[seedData];
meta = DSMetaSeedRange[groups, {a[v1], a[v2], b[e1], b[e2]}];
```

若传入flat allSeeds, 整体统计为一组; 若传入nested seeds, 则每个顶层元素为一组, 组内任意深度先完全Flatten。因此用户可自行组织seeds, 不要求使用缺省生成元分组。声明指标相对seed 中的实际变量多报或少报都会双语warning, 但metadata 按实际完整集合继续初始化; 再次调用会覆盖此前状态。

统一最终关系包络写成

```
ibp = DSGenerateIBP[allSeeds, {-1, 1}];
```

表示所有root 连续指标在生成后的IBP 关系中都必须落在整数闭区间 $[-1, 1]$, 不是直接令seed 指标遍历该区间。对某组某指标的shift 集合 Δ , 程序自动使用 $[-1 - \min \Delta, 1 - \max \Delta]$ 反推较窄的seed 点域。逐指标精细包络写成

```
ibp = DSGenerateIBP[
  allSeeds,
  {a[v1], -1, 1}, {a[v2], 0, 1},
  {b[e1], 0, 2}, {b[e2], -1, 1}
];
```

精细形式必须exact cover 模板中出现的全部root 连续指标。未知、遗漏、重复、非整数边界、反向区间以及把 $n[\dots]$ 写进范围都会给红色双语Error。对ISP 指标，自动反推的下界还会与用户target 下界取较大者；因此package 不会为了覆盖升幂项自行向更负方向扩张。用户若在统一或精细形式中显式给出负ISP 下界，该下界仍完整有效，不会触发门禁。指数为0 的ISP 自身求导先精确化为零，不产生由package 自己引入的-1 项。失败输入以相应结构化字段返回：

```
unknownIndices, missingIndices, duplicateIndices
invalidRanges, discreteIndicesInRangeSpec
```

n_i 从来不是连续撒点变量；若外部模板仍含符号 $n[\dots]$ ，接口认为离散态或EOM 阶段未完成并拒绝。成功结果的status 与dSIBPStatus 都是"generated"，并保存

```
templateSetHash, templateIntegrityAudit, continuousIndices
targetEnvelopeRules, derivedSeedRangeAudits
emptySeedRangeGroups, equationCount, equations
```

其中equations 是一维记录列表；每项同时带展开后的equation、原模板ordinal/hash、连续取值规则、sector/source/generator、离散态和eomCanonicalQ/forbiddenNData 证书，而不是失去来源信息的裸等式列表。DSLlinear 直接消费整个成功Association。package 不设置方程数、连续规则数、离散态数或sector 数量上限；任何有限的用户范围都完整生成。运行时间和内存因此由用户所选范围及topology 决定。枚举前逐输入组打印

编号 1 (来源 ...) : {指标,min,max},...

Group 1 (source ...): {index,min,max},...

编号只表示用户输入分组，不假定它一定是一个IBP 算符。若反推区间出现 $\min > \max$ ，warning 会说明目标包络窄于该组shift 跨度，因而不存在能使全部移位后积分留在包络内的seed 点；该组IBP 撒点为空，最终关系集合缺组，不能作为完整约化系统。该组保存在emptySeedRangeGroups，相应complete capability 为False。删除、改写或重排任何密封模板都会令templateIntegrityAudit 失败；仅改变外层Table 嵌套而保持flatten 后的同序模板不影响结果。DSMetaSeedRange 是唯一公开的逐组shift metadata 入口；后一次合法调用覆盖最近一次逐组范围状态。

所有运行时info、progress、Warning 与Error 按句先中文、后英文；DSMessagesOff[] 只关闭可选提醒，fatal Error 始终保留。

9.2 timeOnly 的唯一公开表示

DSSeeds 对所有timeOnly family 只返回

```
J[sectorKey_String,timeShifts_List,stateBits_List]
```

并记录representation -> "J[sectorKey,timeShifts,stateBits]"。sectorKey 的第 e 位按root line 顺序取full/shrunk 的1/0；top 是全1 字符串，前导零不可丢失。stateBits 的registry 同样按root line 顺序冻结：massive line 每端点一位，massless quotient 整边共享一位，cross/exponential 与shrunk line 不留占位。这条路线没有momentum generator、 b/bS 或ISP，但fixed-line 模长幂仍在sectorPrefactorData 中参与ds 的乘积法则。用户不输入也不能调用Private line pack 或vertex basis。

例如atomic massless 同分支family 的公开流程为

```
context = DSInit[case, RegisterAsCurrent->False];
seeds = DSSeeds[context];
linear = DSLinear[seeds, context];
```

其中top sector 的公开shared state 为 $\{0,1\}$; raw $(n_1, n_2) \in \{0,1\}^2$ 只用于quotient 与导数审计。同分支单线可达sector keys 为 $\{ "1", "0" \}$, 异分支只有"1"。两个顶点各自的time generator 作用于top representatives; 同分支shrink sector 另有合并顶点的time seed。成功时DSSeeds 与DSLlinear 都返回status -> "generated"。含massless full line 的DSTreeSeeds/repIterative/DSTreeNaiveIBP/DSTreeNaiveDE/DSTreeDLogDE 当前返回PendingRederivation 与masslessQuotientFormulaNotCertified; 表示adapter 不绕过这项数学门禁。

linearData 是后端中立的Mathematica Association, 不是reduction 结果。核心字段为:

```
linearEquations, integralList, integralRules
sectorMetadataList, readiness reports
```

全sector 主积分候选必须统一排序, 不能按sector 依次追加。

serializer 只转换linearData 的编号和语法。Kira serializer 默认只写基础输入、映射和metadata, 包括ibp.kira、list 与jobs.yaml。它不保存本机Kira/Fermat 路径, 也不运行Kira。

连续指标范围不属于topology。DSSeeds 只生成general templates; 统一或逐指标范围都在DSGenerateIBP 调用中显式给出。新公开闭环只使用以下接口:

```
context = DSInit[case];
templateData = DSSeeds[context];
allSeeds = DSAllSeeds[templateData];
ibpData = DSGenerateIBP[allSeeds, {-1,1}];
linearData = DSLinear[ibpData, context,
  LinearSystemMode -> "symbolic"
];
```

普通batch 与allSeeds 都完整遍历独立离散基底; masslessFull 的raw 四态审计与source representatives 计数分开保存。DSSeeds 的每条模板保存独立templateHash; DSGenerateIBP 的产物合同包含sealedQ/sourceDigest/completeSystemQ/subsetQ/auditLevel/capabilities。密封完整系统可直接进入formal 后端流程; 密封子集和raw 输入可用于局部线性检查, 但completeSystemQ=False, 缺省DSKiraExport 与formal DSKiraPlan 必须拒绝。seed 或裸模板不能直接交给serializer; 必须先得到成功的backend-neutral linearData。

缺省AuditLevel->"standard" 只消费同源sealed producer 的状态、计数、摘要和digest 字段, 不重算模板/关系内容hash, 也不重复canonical、parity、coverage 或tree residual 扫描。需要开发、独立检验或发布证书时显式使用AuditLevel->"full"。这不会放宽用户输入、未推导公式、递推终止、外部artifact 身份或reduction/DE closure 边界。

9.3 Kira 两阶段计划

DSKiraPlan[linearData,spec] 只在内存中构造计划并返回, 成功状态为"planned"; 它不运行Kira, 也不写任何文件。计划里记录的outputDirectory 只是传给后

续步骤的参数，实际写出Kira 输入目录的是DSKiraExport: 它既接受该计划 (DSKiraExport[plan])，也接受backend-neutral linearData。spec 必须显式给出"stage", 并可使用

```
preferredIntegrals, candidateIntegrals, activeBasis
numericStage, coefficientRules, outputDirectory, jobOptions
```

积分既可写成J, 也可写成当前linearData["integralList"] 的一基编号; 未知项会被丢弃, 若候选因此为空则失败。

linearData["integralList"] 是唯一积分顺序。plan 和exporter 都不会再次排序; 如需改变编号, 必须在构造用户basis 前显式调用一次

```
linearData = DSReorderIntegrals[linearData, orderedJOrIDs];
```

该调用会同步积分表、映射和方程ID。preferredIntegrals 只记录候选偏好, 不改变编号。

预约化阶段按既定顺序冻结用户指定的候选targets:

```
externalRoot = Environment["DSIBP_KIRA_WORKSPACE"];
preWorkspace = FileNameJoin[{externalRoot,"pre_reduction"}];
```

```
prePlan = DSKiraPlan[linearData, <|
  "stage" -> "preReduction",
  "preferredIntegrals" -> preferred,
  "candidateIntegrals" -> candidates,
  "coefficientRules" -> {},
  "outputDirectory" -> FileNameJoin[{preWorkspace,"kira"}],
  "jobOptions" -> Automatic
|>];
preExport = DSKiraExport[prePlan];
```

成功的prePlan 状态为status->"planned", 主要字段是

```
integralOrder, orderingConvention, preferredIntegrals
targetIntegrals, targetCount, numericStage, phaseIsolation
```

preExport 成功时状态为status->"ready", 并附带dSIBPExportManifest 与实际写出的dSIBPExportManifestPath。预约化结果用于外部backend 识别候选masters; 它不是正式DE reduction。

用户取得并检查预约化master 后, 用package 建立有序userMI basis:

```
linearWithUserMI = DSUserMI[linearData,userExpressions,<|
  "names" -> activeNames,
  "activeIndices" -> activeIndices,
  "derivativeVariables" -> {ss11,P0},
  "scalingDegrees" -> activeDegrees
|>];
userMIData = DSUserMI[linearWithUserMI];

formalWorkspace = FileNameJoin[{externalRoot,"formal_reduction"}];
```

```
formalPlan = DSKiraPlan[linearWithUserMI, <|
  "stage" -> "formal",
  "activeBasis" -> Automatic,
  "numericStage" -> "symbolic",
  "coefficientRules" -> {},
  "outputDirectory" -> FileNameJoin[{formalWorkspace,"kira"}],
  "jobOptions" -> Automatic
|>];
formalExport = DSKiraExport[formalPlan];
```

上述两个workspace 都必须位于当前Git 仓库之外；Kira 在WSL 中进入对应目录运行。仓库examples 不保存这两个工作树或它们的中间产物。userExpressions 的每一项可为单个J 或齐次J 线性组合。DSUserMI 检查全体/active 秩，在实际support 内保存userMI[i]->J 与pivot J->userMI[i]+spectator J 的可逆规则；它不把较小basis 宣称为全局积分空间的方阵变换。basis 附加后不能再重排。formal 阶段先对每个active expression 解析构造一阶导数和全部derivative target closure；只有闭合成功才返回

```
activeBasisPreview, analyticDerivativeCertificate
targetIntegralIDsPreview, minimalTargetsQ -> True
```

numericStage 缺省为"symbolic"。若显式选择"postDerivative"，数值规则也只能在解析导数及closure 已冻结后作用；formal plan 会冻结并实际复用同一份preparedLinearData，不得在plan 与export 之间重建另一份线性系统。export manifest 会保留解析证书、数值规则审计，以及equations/map/targets/rules/active payload 和实际导出文件的SHA-256 artifactIdentity；DSKiraImport 按内容身份复核，不能用相同版本字符串替代artifact identity。两个阶段必须使用不同输出目录，不能让预约化manifest、targets 或reduction 覆盖正式结果。active-basis 导数内部始终把显式系数与裸J 分开；EOM、顶点/指标移位和canonical 只作用于裸积分，再乘回系数。若导出前仍残留内部执行helper，DSKiraExport 会拒绝写出，避免manifest 在保存和重新读取时发生二次求值。

稳定失败原因如下；任何失败计划都不能传给DSKiraExport:

```
notLinearData, invalidStage, invalidOutputDirectory
emptyCandidateIntegrals, missingActiveBasis
activeBasisClosureFailed, invalidNumericStage
```

DSKiraExport[plan] 仍会执行seed、linearData、active basis、targets 与微分变量数值化的全部serializer 门禁；成功只表示输入已写好或内存数据已就绪，不表示external reduction 已运行。

10 标准package 与初始化

一个example 的开头分为四个独立单元：package 加载、详细输入、缺省option、初始化。版本目录内Examples 通过公共loader 调用标准Needs 入口：

```
exampleDir = DirectoryName[$InputFileName];
```

```

Get[FileNameJoin[{exampleDir,"..","load_current_package.wl"}],
  CharacterEncoding->"UTF-8"];

(* 详细输入: vertices, lines, momenta, ISP, zero point,
   symmetry, parity 和 independent variables。 *)
case = <| ... |>;

(* 缺省选项: WriteInitializationFiles->False;
   GenerateDerivativeMetadata->False; OverwriteInitialization->False。 *)

init = DSInit[case,
  InitializationDirectory->FileNameJoin[{exampleDir,"init"}],
  WriteInitializationFiles->True,
  GenerateDerivativeMetadata->True
];

```

init/ 中的 manifest.wl、topology.wl、sectors.wl 和 conventions.wl 是可重新 Get 的 Association。derivatives.wl 缺省不生成；只有显式打开 GenerateDerivativeMetadata 时才写出。已有目录若对应不同输入哈希，缺省拒绝覆盖。

初始化还会检查时间 IBP 的小 t 端点。程序在全部 contact-reachable sector 中逐顶点使用已知 h/H 领头幂次表，枚举 endpoint 的 $n = 0, 1$ 与两支行为；任一 Gamma 参数 α 可被精确证明为整数时，未正规化 family 会被拒绝，提示中列出原始顶点编号和 α 。正整数同样拒绝，因为整数指标移位最终会到达非正整数极点。此时应在相应 `a0[vertex]` 中加入 generic 符号正规化参数后重新初始化；程序不尝试用 normalization 或其它发散项做形式抵消。含 generic regulator 的表达式保持未定并允许继续。

DSSeeds 始终完整枚举物理独立的离散基底：massive line 为端点 $n_i = 0, 1$ 四态，massless Full quotient 为 shared bit 的二态；raw 四态 relation 仍可由 metadata 与导数检查审计。连续指标仍保持符号，只有后续 DSGenerateIBP 的显式范围才生成整数点。timeOnly 与 full 共用 Head J，但使用式 (4.2) 与式 (4.1) 两个不兼容 schema；不存在需要用户填写的 TreeState 或其它私有分派参数。

初始化后可查看当前摘要、完整 context 和公开接口清单：

```

DSInfo[]
DSInfo[context,"Full"]
DSPublicAPI[]

```

DSPublicAPI[] 返回当前版本、按工作流分组的全部公开函数和高层函数的实际 Options；它是本附录汇总表与成品 example coverage manifest 的共同核对入口。用户代码不应调用手册未列入该清单的私有 helper。

11 运行消息与进度

package 使用一个集中消息层，所有耗时模块只上报结构化事件，不在内部循环散落 Print：

```
DSMessagesOn[]
DSMessagesOff[]
DSMessagesQ[]
```

缺省开启info、progress 和非致命warning。关闭后隐藏这三类提醒，但fatal error 永远不能静默；函数返回值中的status/issues 仍是权威状态。warning 在notebook 中使用橙色，error 使用红色粗体，同时都发出标准Mathematica Message，使headless 日志也能捕获。

sector 初始化、偏导算符生成、seed、linearization、Kira 导出/导入验证、逐master/变量构造DE 和scaling check 都应给出阶段开始/结束信息。

Notebook 前端使用Monitor 配合ProgressIndicator[n,{0,m}]，或用单个PrintTemporary 配合Dynamic，在原位显示n/m。没有FrontEnd 时只打印开始、结束及大约每10% 的稀疏里程碑，禁止每处理一项增加一行。总数不能预先可靠获得的阶段只报告阶段状态，不伪造n/m。

12 Kira 结果取回到微分方程

package 只生成Kira 输入，不运行reduction。完整闭环为：

```
externalRoot = Environment["DSIBP_KIRA_WORKSPACE"];
workspace = FileNameJoin[{externalRoot,"pure_massive_bubble"}];

templateData = DSSeeds[init];
allSeeds = DSAllSeeds[templateData];
ibpData = DSGenerateIBP[allSeeds,{-1,1}];
linear = DSLinear[ibpData,init];
linear = DSUserMI[linear,userExpressions,<|
  "names"->activeNames,
  "activeIndices"->activeIndices,
  "derivativeVariables"->{ss11,P0},
  "scalingDegrees"->activeDegrees
|>];

formalPlan = DSKiraPlan[linear,<|
  "stage"->"formal",
  "activeBasis"->Automatic,
  "numericStage"->"symbolic",
  "coefficientRules"->{},
  "outputDirectory"->FileNameJoin[{workspace,"kira"}]
|>];
kiraExport = DSKiraExport[formalPlan];

(* 用户从 WSL 进入同一外置目录运行 Kira。 *)
kiraResult = DSKiraImport[
  FileNameJoin[{workspace,"kira"}],init
];
de = DSDE[kiraResult,{ss11,P0},
```

```

    OutputDirectory->FileNameJoin[{workspace,"results","dlogDE"}]
];
scale = DSScaleCheck[de,<|
  "relation"->"PureMassiveBubble",
  "variables"->{ss11,P0},"weights"->{1,1}
|>];

```

DSIBP_KIRA_WORKSPACE 必须指向本发布仓库之外。04 的脚本会拒绝把Kira 工作树写回仓库。WSL 只运行该外置目录中的jobs.yaml; 完成后仍由Wolfram 从同一目录导入结果。04/06 目录中的kira_input_summary.wl 与kira_result_summary.wl 只保存正式检验的轻量记录, 不是reduction fixture。

DSKiraImport 缺省只从results/Tuserweight/kira_list.m 与results/Tuserweight/masters 读取Kira family 输出; 其它位置必须显式给出KiraReductionFile/KiraMasterFile。它检查export manifest 的内容身份、双向积分映射、master list、targets、系数域和完整reduction rules。任何结构化必需文件缺失、内容篡改或convention/parity 不一致都会阻止DE; kira.log 的完成标记只作warning 诊断, 不单独否定已经闭合的结构化结果。根号代数系数可在Kira 侧映射为小写原子, manifest 保存可逆映射; 导入后恢复原表达式。DSDE 对同序master list 调用ds, 因此动力学量系数和完整sector normalization N_s 的导数都自动包含在内:

$$\partial_x J_s = (\partial_x I_s) N_s + \partial_x \log N_s J_s. \quad (12.1)$$

实现中后一项按同一metadata 计算为D[Log[N_s],x] J_s, 不能只微分裸积分核。约化后的内部kk/ISP 系数还会统一转回初始化声明的根号坐标。rep2Integrand 从同一metadata 把 N_s 乘回具体被积函数; 未约回master 的对象必须在结果中显式列出。

numericStage->"symbolic" 时, 参与微分的变量直到DSDE 完成前都保持符号。numericStage->"postDerivative" 则先冻结解析导数和完整target closure, 再把同一固定有理点代入待约化系数; 它只证明该点的reduction 与DE, 不得写成全变量矩阵证明。最终有理probe 必须先检查全部分母非零。交付的pure-massive-bubble closed-loop example 使用缺省root coordinate $ss11=\text{Sqrt}[sp[k,k]]$ 和derivativeVariables->{ss11,P0}, 不能把旧s11 reduction 作字符串改名后充当新坐标的fresh 结果。

formal DE 若需要下述Kira 虚轴映射, 必须先用符号DSLinear 构造linearData; 待旋转的物理坐标及其从属Gram 原子只能在formal DSKiraPlan 的postDerivative 规则中给值。若这些对象已在DSLinear 的CoefficientRules 中被消去, 导出会明确说明数值时序冲突并停止, 因为此时无法保证方程与manifest 使用同一物理截面。只做普通numeric linear reduction、且不构造formal DE 时, 仍可在DSLinear 代入精确规则。

Kira-only 虚轴运动学convention 只作用于serializer/importer。初始化从vertex-phase 与line metadata 识别顶点相位独立参数和fixed massless propagator 模长, 并在进入Kira 前做 $k \rightarrow -I \, ik$; cycle line 的xi 是积分变量, 不列为外部backend 坐标, 复合对象按结构聚合, 不按符号名字猜角色。fixed 模长不是独立符号时, 程序会提示用KinematicRules 绑定独立坐标并停止导出。数值规则只给ik 精确实有理值。manifest 同时保存物理截面、来源顶点/线、双向映射、 $D.k=I \, D_{ik}$ 和Euler 算符不变性; 公开物理API、IBP 与scaling convention 不改变。若该物理原子还定义了内部Gram 坐标, 程序会先形成物理截面再按当前KinematicRules 重生内部规则。例如 $kk[1,1]=\text{loopScale}^2$ 且backend

loopScale->43/17 时, 物理截面为loopScale->-43 I/17、kk[1,1]->-1849/289; 两者符号不一致时export 不得继续。所有剩余整体虚相位由可逆的逐积分phase gauge 消除; ibp.kira 只允许实有理函数, 出现I、Complex 或dsii 时export 直接失败。初次reduction 探测应按预估master 规模设置有界targets, 而不是选择完整积分表; formal workflow 只选择active basis 与其导数闭包。

DSScaleCheck 验证与reduction 数据范围相符的Euler 恒等式。写

$$\partial_{x_a} \mathbf{I} = A_a \mathbf{I} + \mathbf{s}_a, \quad \mathcal{E} = \sum_a w_a x_a \partial_{x_a}, \quad \mathcal{E} I_i = \delta_i I_i, \quad (12.2)$$

DSScaleCheck 根据manifest 物理规则实际固定的scaling variables 数量报告证书范围: 固定零个时为symbolic, 固定全部时为exactPoint, 只固定一部分时为fixedSection; 只有第一种返回symbolicQ=True。因此postDerivative 若只固定 d, ν 等非scaling 参数, 仍可返回全符号matrixResidual/sourceResidual; 若scaling coordinates 已被固定, 则定点或固定截面矩阵不能外推成全局符号DE。此时全局符号齐次性由active master 的外部Euler 导数与time/q-dilation IBP 恒等式在source level 单独认证, 并与相应residual 分别通过。则同序、逐项齐次的master basis 应满足

$$\sum_a w_a x_a A_a = \text{diag}(\delta_i), \quad \sum_a w_a x_a \mathbf{s}_a = 0. \quad (12.3)$$

若换基为 $\mathbf{I}' = T(x) \mathbf{I}$, 则必须使用

$$A'_a = (\partial_{x_a} T) T^{-1} + T A_a T^{-1}, \quad B' = \mathcal{E}[T] T^{-1} + T B T^{-1}, \quad B = \sum_a w_a x_a A_a. \quad (12.4)$$

因此依赖动力学量的normalization 不能只作矩阵共轭; $\mathcal{E}[T] T^{-1}$ 是scaling-degree 修正。对normalized master $M_i = N_i J_i$, 其 δ_i 还必须包含 $\mathcal{E}[N_i]/N_i$ 。reference bubble 中ks*DEks0+P0*DEP00 正是权重{1,1} 的Euler matrix; 应检查非对角元为零、对角元等于同序master degrees, 且结果不依赖整体尺度。

首个闭环example 固定使用pure massive bubble reference 的-- branch 和even-parity closed subsystem, 不生成全parity 大系统。reference 的顶点交换symmetry 只在 $P_1 = P_2$ 成立。reference 的 $V_{\pm} = 0$ 与package 的-- 分支使用相反的能量参数号, 因此 $P_{\text{pkg}} = -P_{\text{ref}}$; Kira-only 截面再取 $P_{0,\text{pkg}} = -i ip0$ 。独立 P_1/P_2 family 不得复用该symmetry。

同目录reference_user_mi_basis.wl 只保存21 个有序候选、前19 个active 位置、physical ks=ss11、导数变量和scaling degrees; main.wl 必须把这些数据交给package DSUserMI, 由package 构造秩、双向映射、backend ID 与derivative closure, 不在example 中自写basis adapter。dlog_basis.wl 只保留reference-readable 记号。当前package fresh Kira 2.3 生成5992 条canonical relations、6012 条formal equations 和2966 个积分, 得到19 个active masters、215 个targets 和0 unreduced; w10*1 的墙钟时间为91.03s。恢复reference 原始dlog basis 第15–18 项的显式 k_s 后, physical P_0 、backend ip0 与physical k_s 三套 19×19 矩阵均为361/361 精确相等、非零差值0。reference 侧只复制并校验既有DEP0.m/DEks.m/DEscaleCheck.m/MidlogNote.m/derivative_rules_bubble.m, 禁止重新生成reference IBP 或运行reference Kira。

13 Tree vertex family

Tree 使用原生time-only 公开对象:

`J[sectorKey,timeShifts,stateBits]`

`timeShifts` 按当前sector 的active vertex-component 顺序保存时间幂次; `stateBits` 按该sector 的registry 保存存活building blocks 的离散态。massive-only 论文公式需要的vertex basis 由package 在Private context 内按sectorKey 构造, 完成迭代或dlog 计算后立即映回上述公开表示; 用户结果、诊断、master list 和linearData 均不得出现一参数或旧line-packed time-only J。

Loop 到tree 的投影使用完整物理线幂。目标项相对参考seed 的每条线显式能量系数为 $k_e^{-\Delta(b+b_0)}$; 缩并线改用 $bS + bS_0$ 。source 与target 都按 $(-\tau)^A$ 定义, contact 合并不另乘 $(-1)^{\Delta\Sigma(a+a_0)}$ 。当前sector 的 a_0 进入tree 的 ν_0 。所以h-mode contact 的 $bS = b + 1$ 、 $bS_0 = b_0 + 2\nu$ 必须共同给出 $k^{-2\nu-1}$, 不能只保留整数 k^{-1} 。

单顶点 p -fold family 的 2^p 个masters 按二进制顺序 $00\dots 0, 00\dots 1, \dots, 11\dots 1$ 保存。`repIterative[expr,end]` 使用2401.00129 Eq. (3.37)、(3.47)、(3.50) 直接生成的general-index 单次规则, 把每个 a_e 约化到指定终点; `Automatic` 表示全零终点, 长度不符或指标/终点不是可判定整数时拒绝。它不设置最大步数, 而逐边验证字典序{剩余可contact 的theta 线数, $\sum_e |a_e - a_{e,\text{end}}|$ } 严格下降, 并检测重复canonical 状态; 不下降、递推奇异或循环才是数学失败。含contact source 时, raw 与sector-tagged 迭代都从DSTreeSeeds 的direct pure-time relation 生成同一首步, 因而保留line momentum 决定的传播子模长; 旧三槽loop 投影只作交叉验证。

DSTreeDLogDE 的sector 对角块使用该文Eq. (3.54)–(3.55), 非对角块使用Eq. (3.66)–(3.68)。letters 按顶点顺序逐块排列: 每个顶点先列massive-leg energies, 再按binary master order 列cut letters, 最后稳定去重; `letterMatrices` 使用同一键顺序。 G^{+-}/G^{-+} 没有theta, 不使用Wronskian/contact; G^{++}/G^{--} 的contact source 先由已验证的loop `dtau`、compiled W_T 、共同-theta和coincident canonical 处理, 再映射到tree lower sector。

程序对每个binary master 保持 n 不变, 只把当前求导顶点设为 $a = 1$, 用所得 $R^{(1)}$ 构造非对角contact selector; 不会误用 $a = 0$ seed 的 $R^{(0)}$ 。每个lower sector 分别保存有理contact selector 与物理sector prefactor, 后者吸收Wronskian 和zero-point 产生的连续能量幂; complete normalization 的对数导数由naive/direct 两条路线共同消费。以下返回字段用于审计:

`masterNormalizationRecords, dlogSelectorCoefficients, selectorAudits, contactMaps`
`omegaBlocks, dlogQ`

`masters` 中每项的coefficient 是对应sector 的有理selector S_s , 不是完整物理normalization; N_s 与乘积 $S_s N_s$ 只在同键的masterNormalizationRecords 中给出。矩阵、letters 与该master 顺序完全一致。

还提供不使用上述直接dlog/递推矩阵的交叉路线:

`ibp = DSTreeNaiveIBP[context, treeDLog["masters"]];`
`de = DSTreeNaiveDE[ibp, {K1,K2,k12}];`

DSTreeNaiveIBP 对每个sector 的binary master 和活动顶点生成 $a_v = 1$ 的loop 代表元, 只调用dtau 与既有contact canonical, 再把投影方程作为有限线性系统直接求解; 它不调

用`repIterative` 或 $A_{\pm}$ 。DSTreeNaiveDE 对外腿相位项复用loop 投影, 对massive-line 动量模长则直接使用 h 的动量导数关系, 因为loop 适配器中的内部动量是积分变量, 不能冒充tree 的外部参数。对于bare sector integral I_s 的endpoint state n , raw tree 导数为

$$\partial_k I_s[a, n = 0] = -I_s[a + 1, n = 1], \quad (13.1)$$

$$\partial_k I_s[a, n = 1] = I_s[a + 1, n = 0] - \frac{2\nu + 1}{k} I_s[a, n = 1]. \quad (13.2)$$

同一传播子的两个endpoint 分别贡献, 随后统一用naive time-IBP 约到指定masters。程序依次使用 $J_s = N_s I_s$ 与 $F_s = S_s J_s$, 所以乘积法则完整加入 $(\partial S_s) J_s$ 、 $S_s (\partial \log N_s) J_s$ 和 $S_s N_s \partial I_s$; 只允许前两项产生参数与动力学量的有理对数导数, 不把 $k^{c\nu}$ 留在系数中。输出的`matrices/sources/residualObjects` 与输入master 顺序一致。缺省masters 来自DSTreeDLogDE; 做独立检验时应显式传入列表, 确保两路线只共享basis 和normalization, 而不共享reduction rules。

若不同lower sectors 具有相同长度的`timeShifts`, 公开`sectorKey` 仍唯一编码shrink set; 内部line-pack pattern 只用于验证该key, 不是第二套公开身份。公式型tree 路线当前只认证massive-only family; 含massless full line 时五个tree 公式接口均fail closed 为PendingRederivation, 直到quotient basis 的递推和dlog 公式重新推导完成。

14 常见错误与当前限制

- 把`sp` 的Orderless 当作图对称性: 错误。
- massless line 交换endpoints 却不翻转 $n = 1$ 表示: 错误。
- massless $n = 1$ theta 导数漏掉 $-2/+2$ shrink: 错误。
- 对 $\delta\theta^m$ 直接代入 $\theta(0)^m = 2^{-m}$: 错误; 必须先合并共同theta 或统一正则化整个乘积。
- massive/massless shrink 都令merged a 减1: 错误; 只有massive Wronskian 有该整数移位。
- 把所有full lines 的幂集当作sector: 错误; 只枚举contact 可达状态。
- top 方程的sub-sector 项仍用source endpoints canonical: 错误, 当前实现已按目标sector 修复。
- massive ++ 与-- 使用同一contour sign, 或让raw contact 与child normalization 使用不同sign: 错误; 两处必须使用同一个由顶点派生的 χ_e 。
- 在EOM 前保留符号 n 或把 $n = 2$ 留给后端: 错误。
- shrink 后仍在 J 保留两个独立 a : 错误。
- 把`ke[i]` 放入momentum generators: 错误。
- seed 直接导出Kira: 错误; 先构造`linearData`。

当前未实现：一般图automorphism/参数对称性的自动发现、scaleless 全自动前端、自动选择propagator basis、streaming/chunked 高圈seed、massless 三槽quotient basis 上的公式型tree 递推/dlog，以及实际运行reduction。当前版本已实现标准context、动力学变量审计、h/H parity、交互消息、Kira 结果取回和loop/tree DE/scaling；package 仍只验证并导入用户在外部分生成的完整reduction。

A 公开函数汇总

函数	用途	主要option/缺省
DSKinematics	提议或审计结构/routing、双动量列表、动力学变量规则、秩、零空间与可逆性	rules=Automatic (缺省提案)
DSInit	解析family、注册context、生成topology/sector/convention metadata	KinematicRules=Automatic; 写初始化文件=False; 写导数算符=False; 覆盖=False
DSInfo	返回当前初始化摘要与readiness	无
DSPParameterNotation	返回缺省/当前根号notation 与用户变量	context=当前已注册context
DSRedefineParameters	用完整新规则重新初始化参数坐标	进度=Automatic
DSMessagesOn, DSMessagesOff, DSMessagesQ	分别开启、关闭或查询非致命提醒; fatal error 始终保留	无
DSPublicAPI	返回公开函数分组、清单与高层缺省options	无
DSSeeds	massive 完整枚举四态、masslessFull 枚举00/10 quotient representatives, 并生成连续指标保持general 的一维模板allSeeds 与推荐分组; 所有模式返回统一三参数J	生成shrink sector=True; 进度=Automatic
DSAllSeeds	从seedData 或最近一次成功调用读取扁平模板列表	无
DSSeedGroups, DSSeedGroupMetadata	读取缺省嵌套seed 分组及其同序来源metadata; 也可读取最近一次成功结果	无
DSMetaSeedRange	按用户seeds 外层结构初始化逐组shift metadata; flat 为一组, nested 按顶层分组; 再次调用覆盖旧状态	声明指标多报或少报只warning, 按实际变量继续
DSGenerateIBP	从统一或逐指标最终关系包络反推逐组seed 点域, 先筛parity 再撒点; 返回目标包络、逐组范围、空组、方程记录和canonical 证书	无数量上限; 进度=Automatic
DSLinear	seed 转后端中立linearData, 其integrallist 是唯一后端顺序	系数规则=Automatic; Kira ordering=Automatic
DSReorderIntegrals	在basis/backend 前显式同步重排积分表、映射与方程ID	order 为现有J 或ID 列表
DSUserMI	从有序J/齐次线性组合构造可逆support 坐标、公开userMI token 与导数闭包	spec 给名称、active 位置、导数变量和scaling 次数
DSKiraPlan	按既定顺序生成预约化candidates, 或formal userMI derivative closure/minimal targets	stage 必填; numericStage=symbolic; 进度=Automatic
DSKiraExport	从linearData 写Kira 基础输入与映射	输出目录=None; active basis=None; job options=Automatic
DSKiraImport	读回完整Kira reduction/master 输出	缺省为results/Tuserweight/family 文件; 其它文件显式指定; 进度=Automatic
DSDE	用ds 与reduction 构造同序DE, 并检查所有导数积分已约回master	变量=初始化全部; 输出目录=None
DSScaleCheck	检查top/residual Euler scaling	关系=Custom; 变量/权重/次数=Automatic
DSTreeSeeds	生成direct pure-time/tree sector rules; loop 投影只作交叉验证	sector=all; 保留source=True

续下页

函数	用途	主要option/缺省
<code>repIterative</code>	<code>tree</code> 积分迭代到指定 a_e 终点; 严格下降/循环/奇异性门禁保证数学终止	终点=全零; 无步数上限
<code>DSTreeNaiveIBP</code>	联立投影后的 <code>tree time-IBP</code> 并解一步升幂对象	<code>masters</code> =直接 <code>dlog</code> 同序列表; 进度= <code>Automatic</code>
<code>DSTreeNaiveDE</code>	用 <code>naive tree IBP</code> 构造同序 <code>normalized-master DE</code>	变量=顶点与 <code>massive-leg energies</code> ; 进度= <code>Automatic</code>
<code>DSTreeDLogDE</code>	生成 <code>tree dlog connection</code> 、 <code>letters</code> 和同序 <code>masters</code>	无
<code>dtau</code>	指定顶点的 <code>time-IBP</code>	显式 <code>topology</code> 或当前 <code>context</code>
<code>dqq/dqk</code>	指定 <code>loop generator</code> 的 <code>momentum-IBP</code>	显式 <code>topology</code> 或当前 <code>context</code>
<code>ds</code>	对带动力学系数的 J 线性组合求总导数	变量必须是初始化外部名字
<code>rep2innerform</code>	用户/内部标量积表示转换, 不改 J 指标; 无唯一逆	显式 <code>topology</code> 或当前 <code>context</code>
<code>rep2outform</code>	映射时 <code>inner</code> 明确失败	
<code>rep2Integrand</code>	把 <code>loop</code> 指标积分展开成惰性被积函数	不积分、不生成 <code>IBP</code>
<code>symmetry</code>	应用自动 <code>tadpole</code> 与用户 <code>symmetry rules</code> 的并集	单次应用, 不自动重复
<code>repSymmetry0</code>	返回输入中的原始用户 <code>symmetry rules</code>	无

表中的原子、`tree` 与`DS*` 高层接口均由当前主线提供。各函数的精确可用`options` 以`Options[函数]` 为准; `massless` 公式型`tree` 接口以结构化`PendingRederivation` 明确失败, 不得静默回退到历史表示或不完整结果。